

Relatório Final do Laboratório 03: Sincronização e Deadlocks

Discente: Pedro Brassi Luccas

RA: 2022.2.08.009

1. Introdução

Este laboratório comparou quatro experimentos: produtor-consumidor, leitores-escretores, jantar dos filósofos e deadlock entre duas threads. A ideia foi ver como o controle de acesso a recursos compartilhados afeta a consistência dos dados, o paralelismo e o progresso das tarefas, relacionando os resultados com os conceitos vistos nas Aulas 10 e 11.

2. Produtor-consumidor

Tabela 2.1. Resultados registrados para o buffer de capacidade 5.

Execução	Versão	Total produzido	Total consumido	Buffer final
1	Sem controle	6032	6315	0
2	Sem controle	6125	6242	0
3	Sem controle	6353	6361	0
4	Sem controle	5829	6119	0
1	Com semáforos	10000	10000	0
2	Com semáforos	10000	10000	0
3	Com semáforos	10000	10000	0
4	Com semáforos	10000	10000	0

Questão 1: Comparação e causa dos resultados

Sem controle, os totais variaram nas quatro execuções, mesmo com o buffer final sempre zerado. A relação esperada ($\text{buffer final} = \text{total produzido} - \text{total consumido}$) não se sustentou: as diferenças foram -283 , -117 , -8 e -290 . Ou seja, dar zero no buffer e não registrar violações da faixa $[0, 5]$ não prova que a execução foi correta.

Em `ProdutorConsumidorSemControle.java`, a sequência `int atual = itensNoBuffer`, teste do limite e escrita de `atual + 1` ou `atual - 1` acontece sem exclusão mútua. Duas threads podem ler o mesmo valor e sobrescrever a alteração uma da outra; por exemplo, dois produtores lendo 2 e ambos gravando 3, perdendo um incremento. O `Thread.yield()` ajuda a intercalar as execuções, mas não sincroniza nada. Os totais também usam incrementos sem proteção, então também podem perder atualizações.

Já em `ProdutorConsumidorComSemaforo.java`, o mutex protege o contador e os totais; espaços bloqueia produtores quando não há vaga; `itens` bloqueia consumidores quando não há item. A espera por vaga ou item acontece antes de pegar o mutex. Assim, cada operação é concluída com segurança: 10000 itens produzidos e consumidos, saldo final zero em todas as rodadas. Essa coordenação bate com os três requisitos da Aula 10, slide 7.

3. Leitores-escritores

Tabela 3.1. Comparação das duas formas de controle de acesso.

Versão	Máx. leitores simultâneos	Espera do escritor	Leituras durante a espera	Tempo total
Mutex simples	1	358 ms	7	1205 ms
ReadWriteLock	8	32 ms	0	512 ms

Questão 2: Paralelismo e exclusividade

A versão `LeitoresEscritoresMutexSimples.java` usa `new Semaphore(1, true)`. Leitores e escritor disputam o mesmo semáforo, que só libera uma permissão. Por isso, só um leitor acessa o recurso por vez, e a escrita também é exclusiva. A fila justa organiza a ordem de atendimento, mas não dá prioridade imediata ao escritor. Na execução registrada, ele esperou 358 ms e sete leitores entraram durante essa espera.

A versão `LeitoresEscritoresComPrioridade.java` usa `new ReentrantReadWriteLock()`. O `readLock()` permite vários leitores ao mesmo tempo, e o `writeLock()` só é liberado quando não há leitor nem outro escritor usando o recurso. Isso explica os oito leitores simultâneos, sem sobreposição entre leitura e escrita. Essa exclusividade vem do próprio código; as saídas não mostram individualmente todos os intervalos de leitura.

Com `ReadWriteLock`, a espera do escritor caiu de 358 para 32 ms (redução de cerca de 91,1%) e o tempo total foi de 1205 para 512 ms, quase 57,5% menor. Esses números valem para essa execução específica, não são garantia de desempenho para qualquer carga de trabalho. Como as leituras simuladas podem se sobrepor, a solução evita a serialização desnecessária discutida na Aula 10, slide 12.

Apesar do nome "ComPrioridade", o construtor usado não ativa nenhuma política formal de preferência para leitores; o lock funciona no modo padrão, não justo. Não há garantia de ordem FIFO. Nessa execução o escritor conseguiu acesso e terminou a escrita, então não houve inanição (starvation) observada. Mas uma única execução não é suficiente para garantir um limite fixo de espera para toda thread.

Interpretação do contador de leituras

O programa imprime "leituras concluídas durante a espera", mas na verdade incrementa `leiturasDuranteEspera` Escritor logo depois de pegar o bloqueio, antes do `sleep(50)`. Ou seja, a métrica conta entradas na leitura enquanto `escritorAguardando` estava verdadeiro; não conta quantos leitores já ativos terminaram durante a espera. Por isso, o zero na versão `ReadWriteLock` não quer dizer que nenhum leitor terminou enquanto o escritor esperava; só quer dizer que nenhuma nova entrada foi contada nesse intervalo.

O ponto central é que as duas soluções protegem a escrita, mas só o `ReadWriteLock` preserva o compartilhamento seguro entre leitores. Sincronizar direito significa escolher a granularidade certa para cada operação, e não bloquear todo acesso concorrente.

4. Jantar dos filósofos

Tabela 4.1. Comportamento registrado com cinco filósofos.

Versão	O que aconteceu	Garfos presos no início	Refeições em 8 s
Sem controle	Avisos de possível deadlock; nenhum filósofo chegou a comer.	5 filósofos segurando um garfo cada.	Não informado
Com saleiro	Os filósofos conseguiram realizar refeições.	Não informado	158

Questão 3: As quatro condições de Coffman

Condição	Sem controle	Com saleiro
Exclusão mútua	Presente. <code>synchronized(garfos[esquerda])</code> e <code>synchronized(garfos[direita])</code> reservam cada garfo para um único filósofo.	Presente. Os mesmos blocos <code>synchronized</code> protegem os garfos; o <code>Semaphore(1)</code> também deixa o saleiro exclusivo.
Posse e espera	Presente. Dentro do bloco do garfo esquerdo, o filósofo pede o outro garfo sem soltar o primeiro.	Presente. Depois de <code>saleiro.acquire()</code> , o filósofo pode esperar por um garfo segurando o saleiro, ou pelo segundo garfo segurando o primeiro.
Não preempção	Presente. Os monitores dos garfos são liberados só ao saírem dos blocos <code>synchronized</code> ; ninguém tira o garfo de quem já o pegou.	Presente. Os garfos continuam sendo liberados só ao saírem dos blocos, e o saleiro volta com <code>saleiro.release()</code> .
Espera circular	Presente no estado observado. Cada filósofo segura o garfo <code>i</code> e espera o garfo $(i + 1) \% 5$, formando um ciclo entre os cinco.	Eliminada. <code>saleiro.acquire()</code> deixa só um filósofo por vez tentando pegar os garfos na fase de aquisição. <code>saleiro.release()</code> só acontece depois de conseguir os dois.

Na versão sem controle, as quatro condições aparecem juntas: F0 espera o garfo de F1, F1 espera o de F2, e assim por diante até F4 esperar o de F0. O registro mostra todos os garfos esquerdos pegos e nenhum direito, seguido de avisos sem progresso. A barreira do código sincroniza a largada antes da aquisição dos garfos, e o estado descrito bate com o que foi observado.

Na solução do saleiro, quem já tem os dois garfos come e os solta sem pedir mais nada. Quem ainda está tentando pegar os garfos é o único nessa fase; os outros esperam o saleiro sem estar com nenhum garfo na mão. Por isso não se forma a cadeia circular de aquisições parciais. Como o saleiro é devolvido antes de comer, filósofos com garfos diferentes podem comer ao mesmo tempo. A solução resolve a espera circular sem abrir mão das outras condições, como mostra a Aula 10, slide 28.

O total positivo mostra que houve progresso, mas não prova equidade entre os filósofos. Além disso, `refeicoes Totais++` é incrementado antes de terminar a refeição e usa um inteiro `volatile`, cujo incremento não é atômico. Então o 158 deve ser lido como o contador que o programa reportou, não como uma medição exata de refeições concluídas.

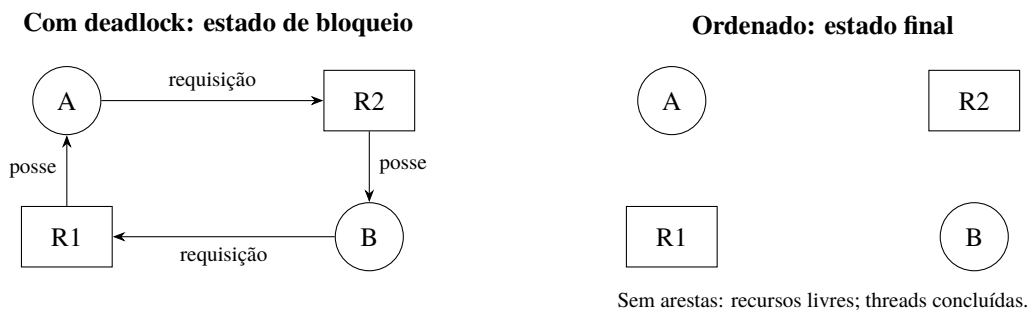
5. Deadlock dirigido e ordenação global

Tabela 5.1. Ordem de aquisição dos recursos por thread.

Versão	Resultado	Ordem A / B	Tempo registrado
DeadlockDuasThreads	Threads travaram; encerramento forçado pelo programa.	A: R1 → R2 B: R2 → R1	5 s
DeadlockPrevenidoPorOrdenacao	Ambas terminaram.	A: R1 → R2 B: R1 → R2	1,023 s (81593 - 80570)

Questão 4: Grafos de alocação de recursos

Círculos representam os processos (threads A e B); retângulos representam os recursos R1 e R2, cada um com uma instância. Setas de posse vão do recurso para o processo; setas de requisição vão do processo para o recurso.



No bloqueio, as arestas são $R1 \rightarrow A$ e $R2 \rightarrow B$ (posse), e $A \rightarrow R2$ e $B \rightarrow R1$ (requisição). O ciclo $A \rightarrow R2 \rightarrow B \rightarrow R1 \rightarrow A$ é o deadlock nesse cenário de recursos com instância única. O grafo representa o travamento antes do `System.exit(0)`, que só encerra a JVM e não resolve a disputa entre as threads.

Na versão ordenada, as duas threads fazem `synchronized(recurso1)` antes de `synchronized(recurso2)`. Enquanto A tem R1, B espera por R1 sem estar com R2; A consegue pegar R2, termina e libera os dois recursos. Só depois B segue. No final, os recursos estão livres e não sobra nenhuma aresta. A ordem global R1 antes de R2 elimina a espera circular, a estratégia de prevenção vista na Aula 11.

6. Conclusão

Ignorar a sincronização pode gerar contagens inconsistentes ou travar completamente o progresso. Sincronizar corretamente tem um custo: no produtor-consumidor, as execuções foram de 2-3 ms sem controle para 130-140 ms com semáforos, mas a versão sem controle rejeita tentativas e perde atualizações, então não é uma comparação de desempenho equivalente. Por outro lado, o `ReadWriteLock` mostra que sincronizar com a granularidade certa preserva o paralelismo. O saleiro e a ordenação global mostram que controlar a ordem de aquisição dos recursos evita deadlocks ao quebrar a espera circular.